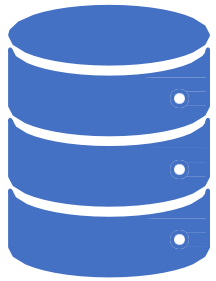
Database Management Systems

UNIT- IV- Transaction and Concurrency Control



Contents:

- Control Transaction management
- ACID Properties
- Serializability
- Concurrency Control(2PL, Deadlocks)
- Time Stamping methods
- Optimistic Methods
- Database Recovery Management

Sub-Modules of Unit 4

Transaction

---What is Transaction

---Transaction Process

i)Read operation ii)Write operation

---Properties of transactions

---Transaction State Diagram

Schedule

i) Serial Schedule ii)Concurrent Schedule

Serializability

i)Conflict Serializability ii)View Serializability

Concurrency Control

- **Need of Concurrency control**
- **Locking Methods** --- Shared Lock, Exclusive Lock
- **Lock Based Protocol** ---Two Phase Locking(2PL)

Strict Two-Phase Locking

Sub-Modules of Unit 4

- **iv) Time Stamping Methods**
- ----Time stamp Based Protocol
- **v) Optimistic Methods**
- ---Optimistic Concurrency control(OCC)
- **vi) Deadlocks**
- ---Prevention ,Detection ,Recovery
- **Database Recovery Management**

Concept Of Transaction Management

- In simple SQL query, the SQL command (DML, DDL etc) sent to server and executed one after another.
- In place of sending one by one SQL command to server we can combine multiple operations that are logically similar and send to server as a single logical unit.
- Example: Transferring Rs 100 from one account to another
  - Withdraw Rs. 100 from account_1
  - Deposit Rs. 100 from account_2
- Collection of multiple operations that form a single logical unit is called as transaction.
- A transaction is a sequence of one or more SQL statements that combined together to form a single logical unit of work.

Concept Of Transaction Management

- **Types of operations** in transactions
  - **Read** – This operation transfers one data item from the database memory to a local buffer of the transaction that executed the read operation. Ex: Data Selection/Retrieval Language
    - SELECT * FROM STUDENTS
  - **Write** – This operation transfers one data item from the local buffer of the transaction that executed the write back to the database. Ex: Data Manipulation Language (DML)
    - UPDATE STUDENT SET NAME='RAVI' WHERE SID=102
- Information processing in DBMS is divided into individual, indivisible operational logical units called transactions.
- Transactions are one of the mechanisms for managing changes to the database
- A transaction is a series of small database operations that together form a single large operation.
- Application programs use transactions to execute sequences of operations when it is important that all operations are successfully completed.

Concept Of Transaction Management

- During the money transfer between two bank accounts it is unacceptable for the operation that updates the second account to fail.
- This would lead to transferred money being lost as it would be withdrawn from one account but not inserted in the second account
  - BEGIN TRANSACTION transfer
    - UPDATE accounts
      SET balance=balance-100
      WHERE account=A
    - UPDATE accounts
      SET balance=balance+100
      WHERE account=B
    - If no errors then
      COMMIT transaction
    - Else
      ROLLBACK transaction
    - End If
  - END TRANSACTION transfer

Transaction Structure And Boundaries

- The transactions consists of all SQL operations executed between the begin transaction and end transaction.
- A transaction is started by issuing a BEGIN TRANSACTION command. Once this command is executed the DBMS starts monitoring the transaction. All operations are executed after a BEGIN TRANSACTION command and are treated as a single large operation.
- When transaction is completed it must either be committed by executing a COMMIT command or rolled back by executing a ROLLBACK command.
- Until a transaction commits or rolls back the database system remains unchanged to other users of the system. Therefore the transaction can make as many changes as it wishes but none of the updates are reflected in the database until the transaction completes or fails.
- A transaction that is successful and has encountered no errors is committed that is all changes to the database are made permanent and become visible to other users of the database.
- A transaction that is unsuccessful and has encountered some type of errors is rolled back that is all changes to the database are undone and the database remains unchanged by transaction.
- The DBMS guarantees that all the operations in the transaction either complete or fail. When a transaction fails all its operations are undone and the database is returned to the state it was in before the transaction started.

Need Of Transaction

- A database management system (DBMS) should be able to survive in a system failure conditions that is if a DBMS is having a system failure then it should be possible to restart the system and the DBMS without losing any of information contained in the database.
- Modern DBMS should not loose data because of system failure.
- A DBMS must manage the data in the database so that as users change the data it does not become corrupt and unavailable.
- Computer systems fail to work correctly for many of below reasons
  - **Software Errors** – Are normally the mistakes in the logic of the program code. Eg when the value of a variable is set to an incorrect value.
  - **Hardware Errors** – They occur when a component of the computer fails for eg a disc drive.
  - **Communication Errors** – In large systems programs may access data stored at numerous locations on a network. The communication links between sites on the network can fail to work.
  - **Conflicting Programs** – When a system executes more than one program at the same time the program may access and change the same data for eg many users may have access to be one set of data.

Need Of Transaction contd..

- Transaction processing is designed to maintain a database in a consistent state.
- It ensures that any operations carried out on the system that is interdependent.
- Transactions are either completed successfully or all aborted successfully.
- Transaction processing is meant for concurrent execution and recovery from possible system failure in a DBMS.

Fundamental Properties (ACID) Of Transaction

	Read (A)
T_i	A := A – 100
	Write (A)
	Read (B)
	B := B +100
	Write (B)

- To understand transaction properties we consider a transaction of transferring 100 rupees from account A to account B as below
- Let T_i be a transaction that transfers 100 from account A to account B. This transaction can be defined as
  - Read balance of account A
  - Withdraw 100 rupees from account A and write back result of balance update
  - Read balance of account B
  - Deposit 100 rupees from to account B and write back result of balance update

Atomicity

- Transaction must be treated as single unit of operation
- That means when a sequence of operations is performed in a single transaction they are treated as a single large operation
- Examples
  - Withdrawing money from the account
  - Making an airline reservation
- The term atomic means things that cannot be divided into parts
- Similarly the execution of a transaction should either be complete or nothing should be executed at all
- No partial transaction executions are allowed

Atomicity Contd..

- Example : Money transfer in above example
  - Suppose some type of failure occurs after Write(A) but before Write(B) then system may loose 100 rupees in calculation which may cause error as sum of original balance(A+B) in accounts A and B is not preserved. In such case database should automatically restore original value of data items
- Example : Making an airline reservation
  - Check availability of seats in desired flights. Airline confirms the reservation. Reduces number of available seats. Charges the credit card of the customer.
- In above case either all above changes are made to database or nothing should be done as half-done transaction may leave data as incomplete state
- If one part of transaction fails, the entire transaction fails, and the database state is left unchanged

Consistency

- Consistent state is a state in which only valid data will be written to the database
- If due to some reasons a transaction is executed that violates the databases consistency rules, the entire transaction will be rolled back and the database will be restored to a state consistent with those rules
- On the other hand if a transaction is executed successfully it will take the database from one consistent state to another consistent state
- DBMS should handle an inconsistency rather than ensure the database is clean at the end of each transaction
- Consistency guarantees that a transaction will never leave your database in a half finished state
- Consistency means if one part of the transaction fails all of the pending changes are rolled back, leaving the database as it was before the transaction was initiated

Consistency Contd..

- Example : Money Transfer in above example
  - Initially the total balance in account A is 1000 and B is 5000 so the sum of balance in both accounts is 6000 and while carrying out above transaction some type of failure occurs after Write (A) but before Write(B) then system may loose 100 rupees in calculation. As now sum of balance in both accounts is 5900 (which should be 6000) which is not a consistent result which introduces inconsistency in database
- Example Student Database
  - When a student record is deleted his details from all the other associated records must get deleted . A properly configured database would not let delete the student record, leaving its associated records stranded
- All the data involved in the operation must be left in consistent state upon completion or roll back of the transaction , database integrity cannot be compromised

Isolation

Isolation property ensures that each transaction must remain unaware of other concurrently executing transaction.

Isolation property keeps multiple transactions separated from each other until they are completed.

Operations occurring in a transaction are invisible to other transactions until the transaction commits or rolls back.

For example when a transaction changes a bank account balance other transactions cannot see the new balance until the transaction commits.

Different isolation levels can be set to modify this default behaviour.

Transaction isolation is generally configurable in a variety of modes. For example in one mode a transaction blocks until the other transaction finishes.

Isolation contd..

- Even though many transactions may execute concurrently in the system. System must guarantee that for every transaction (T_i) other all transactions has finished before transaction (T_i) started or other transactions are started execution after transaction (T_i) finished.
- That means each transaction is unaware of other transactions executing in the system simultaneously.
- Example : Money transfer in above example
  - The database is temporarily inconsistent while above transaction is executing with the deducted total written to A and increased total is not written to account B. If some other concurrently running transaction reads balance of account A and B at this intermediate point and computes A+B it will observe an inconsistent value (Rs. 5900). If that other transaction wants to perform updates on accounts A and B based on inconsistent values (Rs. 5900) that it read, the database may be left in an inconsistent state even after both transactions have been completed.
- A way to avoid the problem of concurrently executing transactions is to execute one transaction at a time.

Durability

Durability property guarantees that the database will keep track of pending changes in such a way that the server can recover from an abnormal termination

Even if the database server is unplugged in the middle of the transaction it will return to consistent state when its restarted

The database handles this by storing uncommitted transactions in a transaction log. By virtue of consistency a partially completed transaction won't be written to the database in the event of an abnormal termination. However when the database is restarted after such a termination it examines the transaction log for completed transactions that had not been committed and applies them

The results of a transaction that has been successfully committed to the database will remain unchanged even after database fails

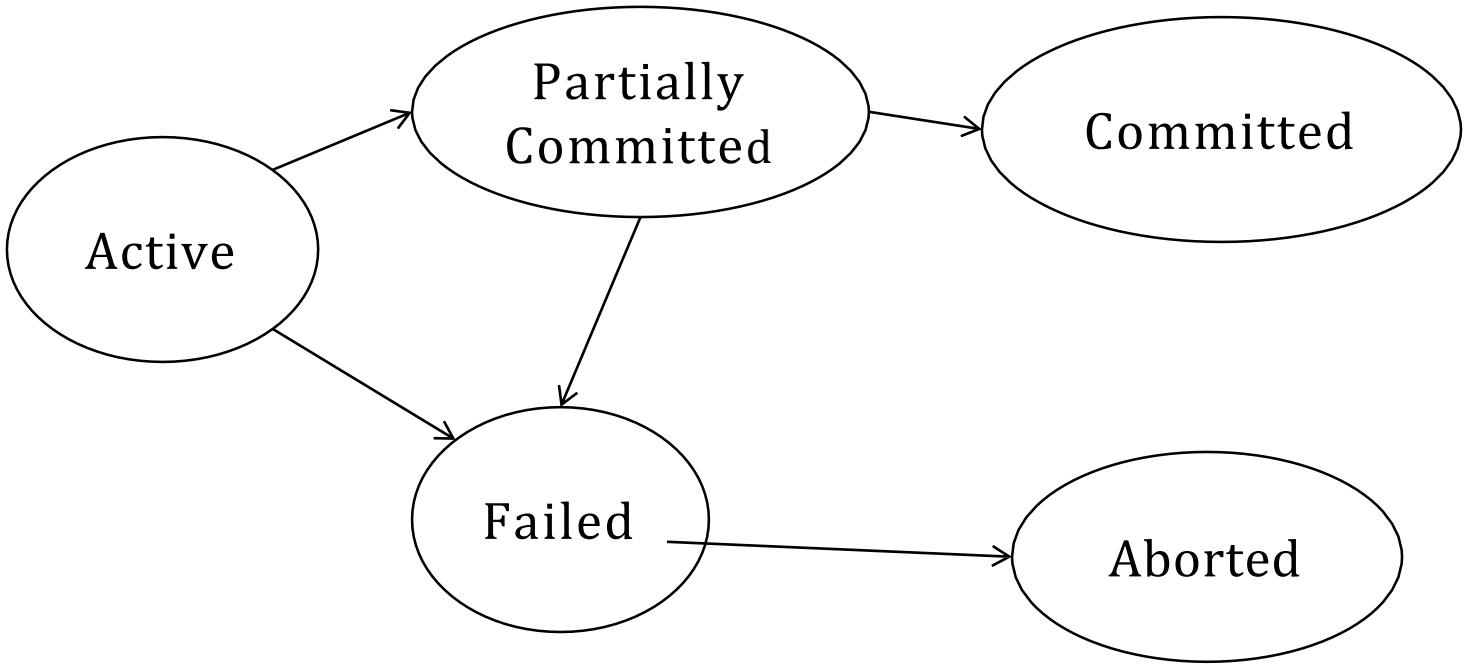
Durability Contd..

- Changes made during the transaction are permanent once the transaction commits
- Example – Once the execution of the transaction completes successfully and the user will be notified that the transfer of the amount has taken place if there is no system failure in this transfer of funds
- The durability property guarantees that once a transaction completes successfully all the changes made by transaction on the database persist even if there is a system failure after the transaction completes execution

Working Of Transaction Processing – Transaction State Diagram

- If a transaction completes successfully it may be saved on to the database server called as committed transaction
- A committed transaction transfers database from one consistent state to other consistent state which must persist even if there is a system failure
- Transaction may not always complete its execution successfully and such transaction must be aborted
- An aborted transaction must not have any change that the aborted transaction made to the database. Such changes must be rolled back
- Once a transaction is committed we cannot undo its effects by aborting it
- A transaction must be in one of the following states

State Diagram Of A Transaction



Working Of Transaction Processing – Transaction State Diagram

- Active
  - This is initial state of transaction execution
  - As soon as transaction execution starts it is in active state
  - Transaction remains in this state till transaction finishes
- Partially Committed
  - As soon as last operation in transaction is executed transaction goes to partially committed state
  - At this condition the transaction has completed its execution and ready to commit on database server, but it is possible that it may be aborted as the actual output may be there in main memory and thus a hardware failure may prohibit its successful completion

Working Of Transaction Processing – Transaction State Diagram

- Failed
  - A transaction enters a failed state after the system determines that the transaction can no longer proceed with its normal execution
  - Example In case of hardware or logical errors while execution
- Aborted
  - Transaction has been rolled back restoring to prior state
  - Failed transaction must be rolled back. Then it enters the aborted state. In this stage the system has two options
    - Restart the transaction – A restarted transaction is considered to be a new transaction which may recover from possible failure
    - Kill the transaction – Because of the bad input or because the desired data were not present in the database an error can occur. In this case we can kill the transaction to recover from failure
- Committed
  - When the last data item is written out, the transaction enters into committed state
  - This state occurs after successful completion of transaction
  - A transaction is said to have terminated if it has either committed or aborted

Transactions Schedules

- Schedule is a sequence of instructions that specify the sequential order in which instructions of transactions are executed
- A schedule for a set of transactions must consist of all instructions present in that transactions it must save the order in which the instructions appear in each individual transaction
- A transaction that successfully completes its execution will have to commit all instructions executed by it at the end of execution
- A transaction that fails to successfully complete its execution will have to abort all instructions executed by transaction at the end of execution
- We denote this transaction T as
  - $R_r(X)$ – Denotes read operation performed by the transaction T on object X
  - $W_r(X)$ – Denotes write operation performed by the transaction T on object X
- Each transaction must specify its final action as commit or abort

Serial Transactions / Schedules

- This is simple model in which transactions executed in serial order that means after finishing first transaction second transaction starts its execution
- This approach specifies first my transaction then your transaction other transactions should not see preliminary results
- Example – Consider below two transactions T1 and T2
- Transaction T_1 – Deposits Rs. 100 to both accounts A and B
- Transaction T_2 – Doubles the balance of accounts A and B

T_1
Read (A)
A = A + 100
Write (A)
Read (B)
B = B + 100
Write (B)

T_2
Read (A)
A = A * 2
Write (A)
Read (B)
B = B * 2
Write (B)

A Consistent Serial Schedule

T_1	T_2	A	B	Operation
		25	25	Initial Balance
Read (A) ; A = A + 100				
Write (A)		125		
Read (B) ; B = B +100				
Write (B)			125	
	Read (A) ; A = A * 2			
	Write (A)	250		
	Read (B) ; B = B * 2			
	Write (B)		250	
		250	250	Final Balance

Concurrent Transactions / Schedules

- Transactions executed concurrently that means operating system executes one transaction for a while then context switches to second transaction and so on
- Transaction processing can allow multiple transactions to be executed simultaneously on database server
- Allowing multiple transactions to change data in database concurrently causes several complications with consistency of the data in the database
- It is very simple to maintain consistency in case of serial execution as compared to concurrent execution of transactions
- Advantages
  - Improved Throughput
    - Throughput of transaction is defined as the number of transactions in a given amount of time
    - If we are executing multiple transactions simultaneously that may increase throughput considerably
  - Resource Utilization
    - It is defined as the processor and the disk performing useful work or not in idle state
    - The processor and disk utilization increases as number of concurrent transaction increases
  - Reduced Waiting Time
    - There may be some small and some long transactions may be executing on a system
    - If transactions are running serially a short transaction may have to wait for a earlier long transaction to complete which can lead to random delays in running a transaction

Concurrent Transaction Example

- Example – Consider below two transactions T1 and T2
- Transaction T_1 – Deposits Rs. 100 to both accounts A and B
- Transaction T_2 – Doubles the balance of accounts A and B

T_1
Read (A)
A = A + 100
Write (A)
Read (B)
B = B + 100
Write (B)

T_2
Read (A)
A = A * 2
Write (A)
Read (B)
B = B * 2
Write (B)

- Above transactions cab executed concurrently as shown below obtained by interleaving the actions of T_1 with those of T_2

A Consistent Concurrent Schedule

T_1	T_2	A	B	Operation
		25	25	Initial Balance
Read (A) ; A = A + 100				
Write (A)		125		
	Read (A) ; A = A * 2			
	Write (A)	250		
Read (B) ; B = B +100				
Write (B)			125	
	Read (B) ; B = B * 2			
	Write (B)		250	
		250	250	Final Balance

Serializability

- The database system must control concurrent execution of transactions which ensure that the database state remains in consistent state.
- We first need to understand which schedules will ensure consistency, and which schedules will not.
- **A schedule is the actual execution sequence of two or more concurrent transactions.**
- A schedule of two transactions T1 & T2 is serializable if and only if executing this schedule has the same effect as either T1:T2 or T2:T1.
- We consider only two operations: read & write.
- Between a read instruction and a write instruction on a data item, a transaction may perform sequence of operations on the copy of that is residing in the local buffer of the transaction.
- Various forms of schedule equivalence are
  - **Conflict Serializability**
  - **View Serializability**

Conflict Serializability

		Transaction Tj	
		Read(D)	Write(D)
Transaction Ti	Read(D)	No Conflict	Conflict
	Write(D)	Conflict	Conflict

- A pair of consecutive database actions(reads, writes) is in conflict if changing their order would change the result of at least one of the transactions.
- Consider schedule S has two consecutive instructions Ii & Ij from transactions Ti & Tj respectively.
- If Ii & Ij access to different data items, then they will not conflict & can be swapped.
- If Ii & Ij access to same data item D then consider following consequences:

Conflict Serializability contd..

- Ii= Read(D), Ij=Read(D) then no conflict as they only read value.
- Ii= Read(D), Ij=Write(D) then they conflict and cannot be swapped.
- Ii= Write(D), Ij=Read(D) then they conflict and cannot be swapped.
- Ii= Write(D), Ij=Write(D) then they conflict and cannot be swapped.
- So we can say that instructions conflict if both consecutive instructions operate on same data item and from different transactions and one of them must be WRITE operation.
- If Ii & Ij access to different data item D then consider following all consequences no conflict as they only read or write different values.

Conflict Serializability contd..

Ii= Read(D)/Write(D), Ij= Read(P)/Write(P) then no conflict as they are only reading or writing different data.

The following set of actions is conflicting:

T1:R(X), T2:W(X), T3:W(X)

While the following sets of actions are not:

T1:R(X), T2:R(X), T3:R(X)

T1:R(X), T2:W(Y), T3:R(X)

View Serializability

- View equivalence is less strict than conflict equivalence, but it is like conflict equivalence based on only the read and write operations of transactions.
- Conditions for view equivalence
  - Schedule S1 and S2 are view equivalent if they satisfy following conditions for each data item D.
    - If Ti reads initial value of D in S1, then Ti also reads initial value of D in S2.
    - If Ti reads value of D written by Tj in S1, then Ti also reads value of D written by Tj in S2.
    - If ti writes final value of D in S1, then ti also writes final value of d in S2.

View Serializability contd..

- First 2 conditions ensure that transaction reads same value in both schedules
- Condition 3 ensures that final consistent state.
- If a concurrent schedule is view equivalent to a serial schedule of same transactions, then it is View Serializable.
- Consider following Schedule S1 with concurrent transactions <T1, T2, T3>
- In both the schedule S1 and a serial schedule S2<T1, T2, T3> reads initial value of D. transaction T3 writes final value of D. So schedule S1 satisfies all three conditions and view serializable to <T1, T2, T3>.

Concurrency Control

In a single user database only one user is accessing the data at any times in multiuse, this means that the DBMS does not have to concern with how changes made to the database will affect other users.

In a multi-user database many users may be accessing the data at the same time. The operations of one user may interfere with other users of the database.

The DBMS uses concurrency control to manage multiuser databases.

Concurrency control is concerned with preventing loss of data integrity due to interference between users in a multiuser environment.

Concurrency control should provide a mechanism for avoiding and managing conflict between various database users operating same data item at same time.

Problems Caused By Concurrent Executions – Lost Update Problem

- Overwrite the update made by another transaction / user

Time	X	Y	Bal
T1		Begin Transaction	100
T2	Begin Transaction	Read (Bal)	100
T3	Read (Bal)	Bal =Bal + 100	100
T4	Bal = Bal – 10	Write (Bal)	200
T5	Write (Bal)	Commit	90
T6	Commit		90

Problems Caused By Concurrent Executions – Lost Update Problem Contd

- Example
- Transaction Y has read the value of bal at time T2 as 100 and transaction X has read the value of bal at time T3 as 100
- At time T4 transaction X has subtracted 10 from its value of bal (200) to disc
- But at time T4 transaction X has subtracted 10 from its value of bal (100) to produce 90
- Transaction X updates the value of bal on the disc at time T5
- The result of this operation is that the update performed by transaction Y (bal + 100 = 200) has been lost
- Transaction X has overwritten the result of transaction Y
- This problem is avoided by not allowing transaction X to read bal until transaction Y has committed its update

Problems Caused By Concurrent Executions – Uncommitted Dependency Problem

- When user sees intermediate step of another transaction before it is committed

Time	X	Y	Bal
T1		Begin Transaction	100
T2		Read (Bal)	100
T3		Bal = Bal + 100	100
T4	Begin Transaction	Write (Bal)	200
T5	Read (Bal)		200
T6	Bal = Bal -10	Rollback	100
T7	Write (Bal)		190
T8	Commit		190

Problems Caused By Concurrent Executions – Uncommitted Dependency Problem - Contd

- Example
- Transaction Y reads and updates the value of bal (100 + 100 = 200) and writes the result at time T4
- At time T5 transaction X reads the value of bal (200) written by transaction Y and updates it
- However at time T6 transaction Y has failed and rolled back
- This means that the update it made to bal is undone and value of bal is returned to100
- Therefore transaction X is updating the incorrect value of bal (200)
- Transaction X should be updating bal = 100 because transaction Y has been rolled back and its changes undone
- Transaction X has used the result of transaction Y but this result was incorrect as transaction Y failed
- This problem is avoided by not allowing transaction X to read the value of bal until transaction Y either commits or rolls back

Problems Caused By Concurrent Executions – Inconsistent Analysis Problem

Transaction reads partial results of incomplete transaction update made by other transaction

Time	X	Y	Bal (X)	Bal (Y)	Bal (Z)	Sum
T1		Begin Transaction	100	50	25	
T2	Begin Transaction	Sum = 0	100	50	25	0
T3	Read (Bal(X))	Read (Bal(X))	100	50	25	0
T4	BaL (X) = Bal (X) -10	Sum = Sum + Bal(X)	100	50	25	100
T5	Write (Bal(X))	Read (Bal(Y))	90	50	25	100
T6	Read (Bal(Z))	Sum = Sum + Bal(Y)	90	50	25	150
T7	Bal(Z) = Bal(Z) + 10		90	50	25	150
T8	Write (Bal(Z))		90	50	35	150
T9	Commit	Read (Bal(Z))	90	50	35	150
T10		Sum = Sum + Bal (Z)	90	50	35	185
T11		Commit	90	50	35	185

Problems Caused By Concurrent Executions – Uncommitted Dependency Problem - Contd

- Example
- Transaction Y is summing the values of Bal (X), Bal (Y), Bal (Z). However at the same time transaction X is transferring 10 pounds between Bal (X) and Bal (Z)
- As transaction Y has used the old balances of Bal (X) and Bal (Z) its final result is incorrect
- This problem would be solved by preventing transaction X from transferring the money between accounts before transaction Y has committed

Concurrency Control Schemes

Fundamental property of transaction is Isolation.

Isolation property ensures that each transaction must remain unaware of other concurrently executing transactions.

When several transactions are executing simultaneously on a database it may not preserve isolation property for a long time.

To implement concurrent system there must be interaction among various concurrent transactions.

This can be done by using one of the concurrency control schemes.

Lock Based Protocols

- **Introduction**
  - In concurrent environment many users can access same data in DBMS simultaneously, each has a feeling of exclusive access to the database.
  - To achieve this system we must have interaction amongst those concurrent transactions.
  - Whenever one transaction is accessing data second transaction should not change data.
  - Transaction can access data if it is not locked by that transaction.
  - Locking is necessary in a concurrent environment to assure that one process should not retrieve or update a record which another process is updating.
  - Failure to this would result in inconsistent and corrupted data.

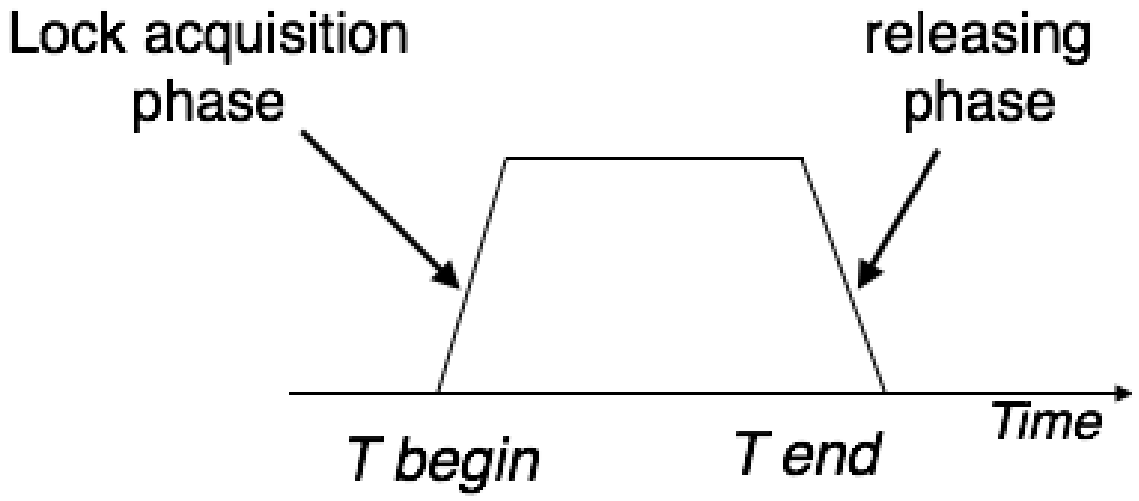
Lock based protocol

Two phase Locking(2PL)

Strict Two-Phase Locking

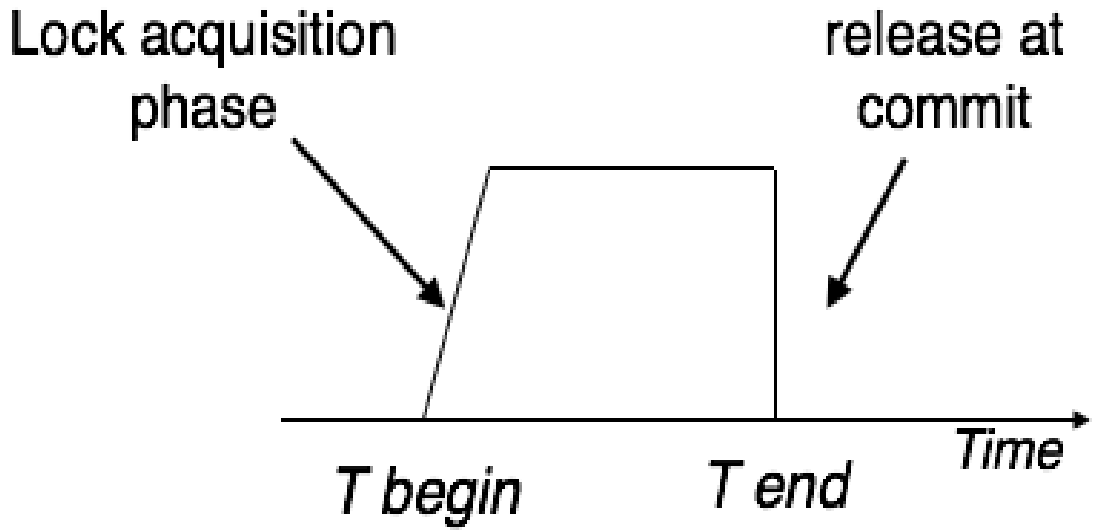
Two phase locking (2PL)

- This locking protocol divides the execution phase of a transaction into three parts.
- In the first part, when the transaction starts executing, it seeks permission for the locks it requires. The second part is where the transaction acquires all the locks. As soon as the transaction releases its first lock, the third phase starts. In this phase, the transaction cannot demand any new locks; it only releases the acquired locks.
- Two-phase locking has two phases, one is **growing**, where all the locks are being acquired by the transaction; and the second phase is **shrinking,** where the locks held by the transaction are being released.
- To claim an exclusive (write) lock, a transaction must first acquire a shared (read) lock and then upgrade it to an exclusive lock.



Strict Two-Phase locking

- The first phase of Strict-2PL is same as 2PL. After acquiring all the locks in the first phase, the transaction continues to execute normally. But in contrast to 2PL, Strict-2PL does not release a lock after using it. Strict-2PL holds all the locks until the commit point and releases all the locks at a time.
- Strict-2PL does not have cascading abort as 2PL does.



Locking Methods:

There are two types of locking to control concurrent access.

1. Shared Lock
2. Exclusive Lock

Shared Lock

- This type of locking is used by the DBMS when a transaction wants to read data without performing modification to it from the database.
- Another concurrent transaction can also acquire a shared lock on the same data allowing the other transaction to read the data.
- Shared locks are represented by S.
- If a transaction T1 has obtained a shared lock on data item X then transaction T1 can only read data item X but cannot write on data item X.
- SQL implementation:

LOCK TABLE Customer IN SHARED MODE

Exclusive Locks

- This type of locking is used by the DBMS when a transaction wants to read or write data in the database.
- When a transaction has an exclusive lock on some data other transaction cannot acquire any type of lock on the data.
- Exclusive locks are represented by X.
- If a transaction T1 has obtained a exclusive lock on data item X then transaction T1 can read data item X and also can write on data item X.
- SQL implementation:

LOCK TABLE Customer IN EXCLUSIVE MODE

Contents

Time stamping methods

Time Stamp Based Protocol

Timestamp Generation

Timestamp ordering algorithm

Time stamp based protocol

Timestamp is a distinct identifier formed by DBMS to recognize the relative starting time of transaction. Generally, timestamp values are assigned to transaction in the order in which transactions are submitted to the system.

A timestamp can be considered as starting time of transaction.

So, time stamping is one of the method of concurrency control in which each and every transaction has a distinct transaction timestamp.

Timestamp generation

- Timestamp can be generated in several ways.
- One way is to assign numbers to each transaction when it is created. For this, counter is used.
- The transaction timestamps are numbered as 1,2,3,…n .
- The counter is reset to zero when no transactions are executing.
- System’s current date/time value can be used to implement timestamps. Its another way to generate a timestamp value.

Timestamp ordering algorithm

- The timestamp-ordering protocol ensures Serializability among transactions in their conflicting read and write operations. This is the responsibility of the protocol system that the conflicting pair of tasks should be executed according to the timestamp values of the transactions.
- The timestamp of transaction T_i is denoted as $TS(T_i)$.
- Read time-stamp of data-item X is denoted by R-timestamp(X).
- Write time-stamp of data-item X is denoted by W-timestamp(X).
- Timestamp ordering protocol works as follows –

Timestamp ordering algorithm

- **If a transaction Ti issues a read(X) operation –**
  - If TS(Ti) < W-timestamp(X)
    - Operation rejected.
  - If TS(Ti) >= W-timestamp(X)
    - Operation executed.
  - All data-item timestamps updated.

- **If a transaction Ti issues a write(X) operation –**
  - If TS(Ti) < R-timestamp(X)
    - Operation rejected.
  - If TS(Ti) < W-timestamp(X)
    - Operation rejected and Ti rolled back.
  - Otherwise, operation executed.

Optimistic methods

- Optimistic Concurrency Control(OCC)
- It is a concurrency control method applied to transactional systems such as relational database management systems.
- OCC is generally used in environments with low data contention. When conflicts are rare, transactions can complete without the expense of managing locks and without having transactions to wait for another transactions locks to clear, leading to higher throughput than other concurrency control methods.
- OCC transaction involves these three phases

1. Begin
2. Modify
3. Validate
4. Commit/Rollback

- Begin: record a timestamp marking the transaction’s beginning.
- Modify: Read database values, and tentatively write changes.
- Validate: Check whether other transactions have modified data that this transaction has used(read or written). This includes transactions that completed after this transaction’s start time and optionally, transactions that are still active at validation time.
- Commit/Rollback: If there is no conflict, make all changes permanent to the database. If there is conflict, resolve it, typically by aborting the transaction.

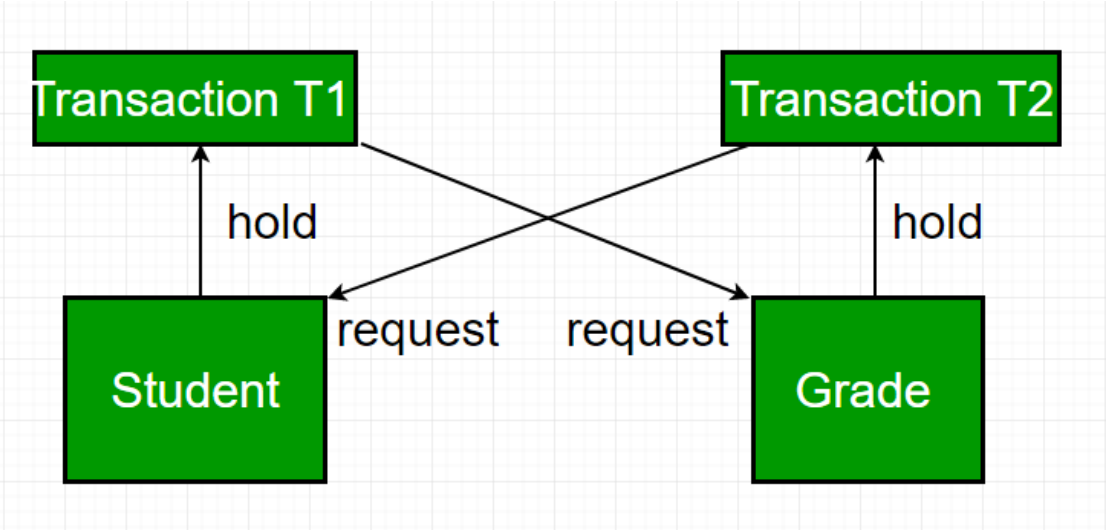
Contents

Deadlock

- In a database, a deadlock is an unwanted situation in which two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as it brings the whole system to a Halt.

- **Example –** let us understand the concept of Deadlock with an example : Suppose, Transaction T1 holds a lock on some rows in the Students table and **needs to update** some rows in the Grades table. Simultaneously, Transaction **T2 holds** locks on those very rows (Which T1 needs to update) in the Grades table **but needs** to update the rows in the Student table **held by Transaction T1**.

- Now, the main problem arises. Transaction T1 will wait for transaction T2 to give up lock, and similarly transaction T2 will wait for transaction T1 to give up lock. As a consequence, All activity comes to a halt and remains at a standstill forever unless the DBMS detects the deadlock and aborts one of the transactions.



Deadlock Prevention

- For large database, deadlock prevention method is suitable. A deadlock can be prevented if the resources are allocated in such a way that deadlock never occur. The DBMS analyzes the operations whether they can create deadlock situation or not, If they do, that transaction is never allowed to be executed.
- Deadlock prevention mechanism proposes two schemes :

  1. Wait-Die Scheme

  2. Wound Wait Scheme

1.Wait-Die Scheme

- In this scheme, If a transaction request for a resource that is locked by other transaction, then the DBMS simply checks the timestamp of both transactions and allows the older transaction to wait until the resource is available for execution.
- Suppose, there are two transactions T1 and T2 and Let timestamp of any transaction T be TS (T). Now, If there is a lock on T2 by some other transaction and T1 is requesting for resources held by T2, then DBMS performs following actions:

- Checks if TS (T1) < TS (T2) – if T1 is the older transaction and T2 has held some resource, then it allows T1 to wait until resource is available for execution. That means if a younger transaction has locked some resource and older transaction is waiting for it, then older transaction is allowed wait for it till it is available. If T1 is older transaction and has held some resource with it and if T2 is waiting for it, then T2 is killed and restarted latter with random delay but with the same timestamp. i.e. if the older transaction has held some resource and younger transaction waits for the resource, then younger transaction is killed and restarted with very minute delay with same timestamp.
  This scheme allows the older transaction to wait but kills the younger one.

2. Wound Wait Scheme

- In this scheme, if an older transaction requests for a resource held by younger transaction, then older transaction forces younger transaction to kill the transaction and release the resource. The younger transaction is restarted with minute delay but with same timestamp. If the younger transaction is requesting a resource which is held by older one, then younger transaction is asked to wait till older releases it.

Contents:

- **Deadlock Detection**

--Wait for Graph

- **Deadlock Recovery**

--Selection of Victim

--Rollback

--Starvation

- **Database Recovery Management**

-- Log Based Recovery

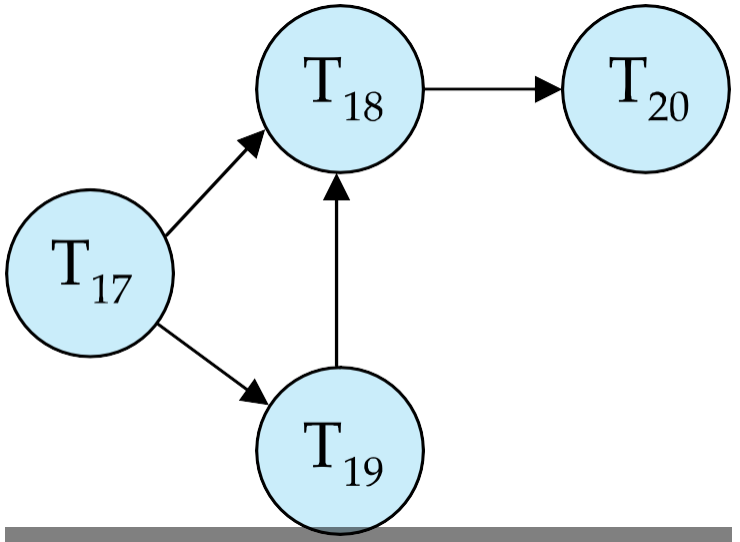
-- Shadow Paging

Deadlock Detection

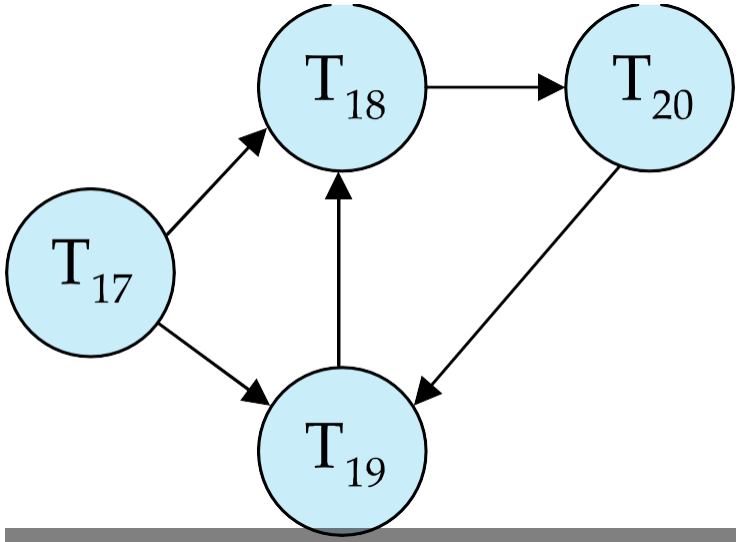
Wait-for Graph

- Deadlocks can be described as a wait-for graph, which consists of a pair G = (V,E)
  - V is a set of vertices (all the transactions in the system).
  - E is a set of edges; each element is an ordered pair Ti $\rightarrow$Tj.
- If Ti $\rightarrow$ Tj is in E, then there is a directed edge from Ti to Tj, implying that Ti is waiting for Tj to release a data item.
- When Ti requests a data item currently being held by Tj, then the edge Ti $\rightarrow$ Tj is inserted in the wait-for graph. This edge is removed only when Tj is no longer holding a data item needed by Ti.
- The system is in a deadlock state if and only if the wait-for graph has a cycle. Must invoke a deadlock-detection algorithm periodically to look for cycles.

Deadlock Detection Contd..



Wait-for graph without a cycle



Wait-for graph with a cycle

Recovery from Deadlock

- When deadlock is detected in the system, then the system should be recovered from deadlock using recovery schemes.
- A most common solution is rollback one or more transaction to break the deadlock.

Methods To Recover from Deadlock

1. Selection of Victim
2. Rollback
3. Starvation

1. Selection of Victim

- If deadlock is detected, then one or transactions are selected to break the deadlock.
- Transactions with minimum cost should be selected for rollbacks.
- Cost can be detected by following factors.
  - For how much time transaction has computed and how much time it needs to do.
  - How many data items it has locked?
  - How many data items it may need ahead?
  - Number of transaction to be rollback?

2. Rollback

- Once victims are decided then there are two ways to do
  - **Total Rollback** – The transaction is aborted and then it is restarted.
  - **Partial Rollback** – Rollback the only transaction which is needed to break the deadlock.
- But this approach needs to record some more information like state of running transactions, locks on data item held by them, and deadlock detection algorithm specifies points up to which transactions to be roll backed and recovery method has to rollback.
- After some time transaction resumes; partial transaction.

3. Starvation

- It may happen every time same transaction is selected as victim and this may lead to starvation of that transaction.
- System should take care that every time same transaction should not be selected as victim. So it will not be starved

Database recovery management

1. Log-Based Recovery
2. Shadow Paging

1. Log based recovery

- Log is a sequence of records, which maintains the records of actions performed by a transaction. It is important that the logs are written prior to the actual modification and stored on a stable storage media, which is failsafe.
- Log-based recovery works as follows –
- The log file is kept on a stable storage media.
- When a transaction enters the system and starts execution, it writes a log about it. $<T_n, Start>$
- When the transaction modifies an item X, it write logs as follows –

$<T_n, X, V_1, V_2>$ It reads T_n has changed the value of X, from V_1 to V_2.

- When the transaction finishes, it logs –

$<T_n, commit>$

The database can be modified using two approaches –

- **Deferred database modification** – All logs are written on to the stable storage and the database is updated when a transaction commits.
- **Immediate database modification** – Each log follows an actual database modification. That is, the database is modified immediately after every operation.

2. Shadow Paging

- *Shadow paging* is an alternative to log-based recovery techniques, which has both advantages and disadvantages. It may require fewer disk accesses, but it is hard to extend paging to allow multiple concurrent transactions.
- The paging is very similar to paging schemes used by the operating system for memory management.
- The idea is to maintain two-page tables during the life of a transaction: the *current* page table and the *shadow* page table.
- When the transaction starts, both tables are identical.
- The shadow page is never changed during the life of the transaction.
- The current page is updated with each **write** operation.
- Each table entry points to a page on the disk.
- When the transaction is committed, the shadow page entry becomes a copy of the current page table entry and the disk block with the old data is released.
- If the shadow is stored in non-volatile memory and a system crash occurs, then the shadow page table is copied to the current page table. This guarantees that the shadow page table will point to the database pages corresponding to the state of the database prior to any transaction that was active at the time of the crash, making aborts automatic.

2. Shadow Paging contd..

- There are drawbacks to the shadow-page technique:
- **Commit overhead.** The commit of a single transaction using shadow paging requires multiple blocks to be output -- the current page table, the actual data and the disk address of the current page table. Log-based schemes need to output only the log records.
- **Data fragmentation.** Shadow paging causes database pages to change locations (therefore, no longer contiguous.
- **Garbage collection.** Each time that a transaction commits, the database pages containing the old version of data changed by the transactions must become inaccessible. Such pages are *garbage* since they are not part of the free space and do not contain any usable information. Periodically it is necessary to find all the garbage pages and add them to the list of free pages. This process is called *garbage collection* and imposes additional overhead and complexity on the system.

Thank You!!!